

COMPLETE REST API SPECIFICATION

BeatPillz Mobile API Documentation

Complete, production-ready REST API guide for mobile engineers building Flutter, React Native, Swift, and Kotlin applications.

PLATFORM VERSION

Laravel 10.x / Sanctum Token Auth

PRODUCTION BASE URL

<https://beatpillz.com/api/v1>

TOTAL ENDPOINTS

35+ Dedicated Mobile Routes

PAYLOAD FORMAT

JSON (RFC 8259)

Table of Contents

1. Overview & Authentication Flow	Sec 1	2. Authentication & Social Sign-In	Sec 2
3. User Profile, KYC & Creator Setup	Sec 3	4. Home Discovery, Categories & Producers	Sec 4
5. Beats Catalog, Previews & Streaming	Sec 5	6. Reviews, Comments & Wishlist	Sec 6
7. Shopping Cart & Multi-License	Sec 7	8. Checkout & 1-Click Wallet Payments	Sec 8
9. User Library, Downloads & Statements	Sec 9	10. Producer Studio & Beat Uploads	Sec 10
11. Subscriptions & Premium Plans	Sec 11	12. Refunds, Support Tickets & FAQs	Sec 12
13. App Config & Multi-Currency	Sec 13	14. Ready Flutter / Dart Client Code	Sec 14

1. Overview & Authentication Architecture

All mobile endpoints are grouped under the prefix `/api/v1/*`. Every request and response uses pure JSON payloads with standardized structure.

Base URLs:

- **Production:** `https://beatpillz.com/api/v1`
- **Local Development:** `http://127.0.0.1:8000/api/v1`

Standard Response Format

All successful responses return HTTP status `200` or `201`:

```
{
  "success": true,
  "message": "Optional descriptive message",
  "data": { ... }
}
```

All validation errors return HTTP status `422`:

```
{
  "success": false,
  "message": "The email has already been taken.",
  "errors": {
    "email": ["The email has already been taken."]
  }
}
```

2. Authentication & Social Sign-In

POST /auth/register

Register new user with name, username, email, password, and optional device name.

```
// Request
{
  "firstname": "John",
  "lastname": "Doe",
  "username": "johndoe",
  "email": "johndoe@example.com",
  "password": "Password123",
  "password_confirmation": "Password123",
  "device_name": "Flutter iOS"
}

// Response (201 Created)
{
  "success": true,
  "message": "Registration successful.",
  "access_token": "1|qWeRtY123456...",
  "token_type": "Bearer",
  "user": {
    "id": 1,
    "firstname": "John",
    "lastname": "Doe",
    "fullname": "John Doe",
    "username": "johndoe",
    "email": "johndoe@example.com",
    "avatar": null,
    "is_author": false,
    "balance": 0.00
  }
}
```

POST /auth/login

Authenticate with **Email** or **Username**.

```
// Request
{
  "login": "johndoe@example.com", // Or "johndoe"
  "password": "Password123",
  "device_name": "iPhone 15 Pro"
}

// Response (200 OK)
{
  "success": true,
  "message": "Login successful.",
  "access_token": "2|aBcDeF654321...",
  "token_type": "Bearer",
  "user": {
    "id": 1,
    "username": "johndoe",
    "email": "johndoe@example.com",
    "balance": 25.50
  }
}
```

POST /auth/social-login

Native Google, Apple, or Facebook mobile sign-in.

```
// Request
{
  "provider": "google", // "google" | "apple" | "facebook"
  "provider_id": "10982347209384",
  "email": "johndoe@gmail.com",
  "firstname": "John",
  "lastname": "Doe",
  "avatar": "https://lh3.googleusercontent.com/...",
  "device_name": "Pixel 8"
}

// Response (200 OK)
{
  "success": true,
  "message": "Social login successful.",
  "access_token": "3|zXcVbN987654...",
  "token_type": "Bearer",
  "user": { "id": 1, "username": "johndoe", "email": "johndoe@gmail.com" }
}
```

POST /auth/forgot-password

Dispatches password reset instructions to user's registered email address.

```
// Request
{ "email": "johndoe@example.com" }

// Response (200 OK)
{ "success": true, "message": "Password reset instructions have been sent to your email address." }
```

POST /auth/logout **TOKEN REQUIRED**

Revokes the active mobile session Bearer access token.

```
// Response (200 OK)
{ "success": true, "message": "Logged out successfully." }
```

3. User Profile, KYC & Creator Setup

GET **/user/profile** **TOKEN REQUIRED**

Returns authenticated user profile, balances, KYC status, author status, and badges.

PUT **/user/profile** **TOKEN REQUIRED**

Update personal info, biography, and social media handles.

```
// Request Body
{
  "firstname": "John",
  "lastname": "Doe",
  "profile_heading": "Producer & Audio Engineer",
  "profile_description": "Crafting hard trap and afrobeat sounds for 5+ years.",
  "social_links": {
    "instagram": "https://instagram.com/johndoe",
    "youtube": "https://youtube.com/@johndoe"
  }
}
```

POST **/user/avatar** **TOKEN REQUIRED**

Upload avatar image file using `multipart/form-data` (key: `avatar` , allowed: jpg, png, webp, max: 4MB).

PUT **/user/password** **TOKEN REQUIRED**

Change password (requires `current_password` , `password` , `password_confirmation`).

GET **/user/kyc** **TOKEN REQUIRED**

Returns KYC verification status (`is_verified`: `true|false`) and submission logs.

POST **/user/become-author** **TOKEN REQUIRED**

Upgrades standard buyer account to Author/Producer status with default level badge.

PUT**/user/withdrawal-account**

TOKEN REQUIRED

Configure author payout account (e.g. `withdrawal_method_id: 1`, `withdrawal_account: "paypal@example.com"`).

4. Home Discovery, Categories & Producers

GET /home

Aggregated mobile dashboard payload with featured beats, trending, best sellers, latest releases, curated categories, and featured producers.

```
// Response Structure
{
  "success": true,
  "data": {
    "featured_beats": [ ... ],
    "trending_beats": [ ... ],
    "best_selling_beats": [ ... ],
    "latest_beats": [ ... ],
    "categories": [ ... ],
    "featured_producers": [ ... ]
  }
}
```

GET /categories

Nested list of all music categories / genres and subcategories with item counters.

GET /producers/{username_or_id}

Public producer profile with their uploaded beats catalog, badges, follower count, and `is_following` flag.

POST /producers/{id}/follow **TOKEN REQUIRED**

Follow or unfollow a producer.

```
// Response
{ "success": true, "is_following": true, "message": "Following producer." }
```

GET /user/following **TOKEN REQUIRED**

Returns list of all producers followed by the user.

5. Beats Marketplace Catalog & Streaming

GET /items

Search, filter, and paginate the marketplace catalog.

Parameter	Type	Example	Description
q	string	?q=afrobeat	Search keyword in title, description, tags
category	string	?category=afrobeats	Category slug or ID
subcategory	string	?subcategory=afropop	SubCategory slug or ID
author	string	?author=metrobeatz	Filter beats by author username
min_price	float	?min_price=10	Minimum price filter
max_price	float	?max_price=100	Maximum price filter
is_free	int	?is_free=1	Filter free beats
is_premium	int	?is_premium=1	Filter subscription premium beats
sort	string	?sort=popular	latest , popular , price_low , price_high , rating
page	int	?page=2	Pagination page

GET /items/{slug_or_id}

Full beat details with streamable audio preview URL, regular/extended prices, discounts, producer profile, and related recommendations.

```
// Response Structure
{
  "success": true,
  "item": {
    "id": 12,
    "name": "Lagos Night (Afropop Beat)",
    "slug": "lagos-night-afropop-beat",
    "description": "Premium Afrobeat instrumental with drums, horns and lead guitar.",
    "preview_audio_url": "https://beatpillz.com/storage/previews/audio/track12.mp3",
    "thumbnail_url": "https://beatpillz.com/storage/thumbnails/art12.jpg",
    "price": {
      "regular": 29.99,
      "extended": 99.99,
      "has_discount": true,
      "discount_regular": 19.99,
      "discount_percent": 33
    },
    "is_free": false,
    "is_premium": false,
    "is_favorited": true,
    "total_sales": 87,
    "avg_reviews": 4.8,
    "category": { "id": 1, "name": "Afrobeats", "slug": "afrobeats" },
    "author": { "id": 4, "name": "Producer X", "username": "producerx" },
    "tags": ["afrobeat", "burnaboy", "wizkid", "drums"]
  },
  "related": [ ... ],
  "author_more": [ ... ]
}
```

GET /items/{id}/download-free

Stream or download free beat files directly without going through cart or checkout.

6. Reviews, Comments & Wishlist

Method	Endpoint	Auth	Description
GET	/items/{id}/reviews	Public	List star ratings & customer reviews for a beat
POST	/items/{id}/reviews	TOKEN	Submit review (rating: 1-5, subject, body). Must have purchased beat.
GET	/items/{id}/comments	Public	Get threaded discussion comments and producer replies
POST	/items/{id}/comments	TOKEN	Post a comment / question on a beat (body)
POST	/items/{id}/favorite	TOKEN	Toggle beat in user's favorites / wishlist
GET	/favorites	TOKEN	List all favorited beats in user wishlist

7. Shopping Cart & Multi-License

GET /cart TOKEN REQUIRED

Get active cart items, license types (Regular vs Extended), and subtotal.

POST /cart/add TOKEN REQUIRED

Add beat to cart with selected license type.

```
// Request Body
{
  "item_id": 12,
  "license_type": 1 // 1: Regular License, 2: Extended License
}
```

DELETE /cart/{id} TOKEN REQUIRED

Remove single item from cart.

DELETE /cart/clear TOKEN REQUIRED

Empty user's entire cart.

8. Mobile Checkout & 1-Click Wallet Payments

GET /checkout/gateways **TOKEN REQUIRED**

Returns list of active payment gateways (Stripe, PayPal, Razorpay, etc.) and current user wallet balance.

POST /checkout/create-transaction **TOKEN REQUIRED**

Converts cart into an unpaid transaction order.

```
// Response (201 Created)
{
  "success": true,
  "transaction_id": 42,
  "total_amount": 29.99,
  "user_balance": 50.00,
  "can_pay_balance": true
}
```

POST /checkout/pay-with-balance **TOKEN REQUIRED**

Instantly completes purchase and delivers items to User Library using account balance.

```
// Request Body
{ "transaction_id": 42 }

// Response (200 OK)
{
  "success": true,
  "message": "Purchase completed successfully!",
  "new_balance": 20.01,
  "transaction_id": 42
}
```

9. User Library, Downloads & Statements

GET /user/purchases **TOKEN REQUIRED**

List all purchased beats in User Library with purchase certificate codes, license types, and review status.

GET **/user/purchases/{id}/download** **TOKEN REQUIRED**

Direct binary stream download for purchased beat audio, lossless WAV files, stems, and licenses.

GET **/user/statements** **TOKEN REQUIRED**

Paginated financial statements and order invoices.

10. Producer Studio & Beat Uploads

GET **/author/dashboard** **TOKEN REQUIRED**

Producer earnings, total sales amount, balance, followers count, and active/pending item counts.

GET **/author/items** **TOKEN REQUIRED**

List all beats uploaded by the producer with review status (1: Pending , 2: Soft Rejected , 4: Approved).

POST **/author/items/upload** **TOKEN REQUIRED**

Upload a new beat directly from mobile using `multipart/form-data` .

Field	Type	Required	Description
name	string	Yes	Title of the beat (max 150 chars)
category_id	int	Yes	Genre / Category ID
sub_category_id	int	No	SubCategory ID
description	string	Yes	Beat description / BPM / Key info
regular_price	float	Yes	Standard MP3/WAV license price
extended_price	float	No	Unlimited / Trackout price
tags	string	No	Comma-separated tags (e.g. trap,808,drake)
preview_audio	file	No	MP3/WAV preview audio file
thumbnail	file	No	Cover artwork image

DELETE **/author/items/{id}** **TOKEN REQUIRED**

Delete draft or rejected beat.

GET **/author/sales** **TOKEN REQUIRED**

Detailed sales breakdown with buyer details and net author commission earnings.

GET**/author/withdrawals**

TOKEN REQUIRED

List available payout methods, minimum thresholds, and withdrawal history.

POST**/author/withdrawals/request**

TOKEN REQUIRED

Submit payout request for producer balance.

11. Subscriptions, Blog & Refunds

Method	Endpoint	Auth	Description
GET	/plans	Public	List active subscription plans & features
GET	/user/subscription	TOKEN	Check user's current subscription & expiry date
GET	/blog	Public	List published blog articles (filter by category)
GET	/blog/{slug}	Public	Get full article details
GET	/refunds	TOKEN	List buyer and producer refund requests
POST	/refunds	TOKEN	Submit refund request (purchase_id, reason, message)
GET	/refunds/{id}	TOKEN	View refund conversation thread & replies
POST	/refunds/{id}/reply	TOKEN	Reply in refund discussion
GET	/tickets	TOKEN	List customer support tickets
GET	/tickets/categories	TOKEN	Get support ticket categories
POST	/tickets	TOKEN	Create support ticket (category_id, subject, message)
GET	/tickets/{id}	TOKEN	View ticket conversation thread
POST	/tickets/{id}/reply	TOKEN	Reply to support ticket
GET	/help/categories	Public	Help center categories & articles
GET	/help/faqs	Public	Platform Frequently Asked Questions
GET	/config	Public	Currencies, rates, legal links, support contacts

12. Complete Flutter / Dart Service Boilerplate

```
import 'dart:convert';
import 'package:http/http.dart' as http;
import 'package:flutter_secure_storage/flutter_secure_storage.dart';

class BeatPillzApi {
  static const String baseUrl = 'https://beatpillz.com/api/v1';
  static const _storage = FlutterSecureStorage();

  // Authentication Headers
  static Future<Map<String, String>> getHeaders() async {
    final token = await _storage.read(key: 'auth_token');
    return {
      'Content-Type': 'application/json',
      'Accept': 'application/json',
      if (token != null) 'Authorization': 'Bearer $token',
    };
  }

  // 1. Email/Username Login
  static Future<Map<String, dynamic>> login(String login, String password) async {
    final res = await http.post(
      Uri.parse('$baseUrl/auth/login'),
      headers: {'Content-Type': 'application/json', 'Accept': 'application/json'},
      body: jsonEncode({'login': login, 'password': password, 'device_name': 'Flutter App'}),
    );
    final data = jsonDecode(res.body);
    if (res.statusCode == 200 && data['success'] == true) {
      await _storage.write(key: 'auth_token', value: data['access_token']);
    }
    return data;
  }

  // 2. Google / Apple Social Sign-In
  static Future<Map<String, dynamic>> socialLogin({
    required String provider,
    required String providerId,
    required String email,
    String? firstname,
    String? lastname,
    String? avatar,
  }) async {
    final res = await http.post(
      Uri.parse('$baseUrl/auth/social-login'),
      headers: {'Content-Type': 'application/json', 'Accept': 'application/json'},
      body: jsonEncode({
        'provider': provider,
        'provider_id': providerId,
        'email': email,
        'firstname': firstname,
        'lastname': lastname,
        'avatar': avatar,
        'device_name': 'Flutter App',
      }),
    );
    final data = jsonDecode(res.body);
    if (res.statusCode == 200 && data['success'] == true) {
      await _storage.write(key: 'auth_token', value: data['access_token']);
    }
  }
}
```

```

    }
    return data;
}

// 3. Home Feed
static Future<Map<String, dynamic>> getHome() async {
    final headers = await getHeaders();
    final res = await http.get(Uri.parse('$baseUrl/home'), headers: headers);
    return jsonDecode(res.body);
}

// 4. Search Beats Catalog
static Future<Map<String, dynamic>> getBeats({
    String? query,
    String? category,
    String sort = 'latest',
    int page = 1,
}) async {
    final headers = await getHeaders();
    final uri = Uri.parse('$baseUrl/items').replace(queryParameters: {
        if (query != null) 'q': query,
        if (category != null) 'category': category,
        'sort': sort,
        'page': '$page',
    });
    final res = await http.get(uri, headers: headers);
    return jsonDecode(res.body);
}

// 5. 1-Click Wallet Checkout
static Future<Map<String, dynamic>> payWithWallet(int transactionId) async {
    final headers = await getHeaders();
    final res = await http.post(
        Uri.parse('$baseUrl/checkout/pay-with-balance'),
        headers: headers,
        body: jsonEncode({'transaction_id': transactionId}),
    );
    return jsonDecode(res.body);
}

// 6. User Purchased Library
static Future<Map<String, dynamic>> getUserPurchases() async {
    final headers = await getHeaders();
    final res = await http.get(Uri.parse('$baseUrl/user/purchases'), headers: headers);
    return jsonDecode(res.body);
}
}

```